

Relatório Técnico: Agente Inteligente para Resolução de Labirintos

Emerson Caetano Ataíde¹, Íthan de Paula Amaral¹, Túlio Vilela Lopes¹

¹ Universidade Federal de Ouro Preto (UFOP)
João Monlevade - MG - Brazil

emerson.ataide@aluno.ufop.edu.br

ithan.amaral@aluno.ufop.edu.br

tulio.lopes@aluno.ufop.edu.br

Abstract. *Este relatório descreve o desenvolvimento de um agente inteligente capaz de resolver labirintos discretos em três cenários distintos: mapa conhecido (busca clássica), labirinto com pontos de coleta (busca local) e ambiente desconhecido (busca online). O projeto envolveu a modelagem do agente via PEAS, a formulação formal dos problemas de busca, a implementação de algoritmos obrigatórios (BFS, DFS, UCS, Gulosa, A*, Hill-Climbing, Simulated Annealing) e uma análise experimental detalhada. O uso de ferramentas de IA generativa foi empregado como suporte ao desenvolvimento, com a devida auditoria incluída. Os resultados demonstram o compromisso entre esforço computacional e qualidade da solução.*

1. Introdução

O objetivo deste trabalho é a modelagem e implementação de um agente inteligente para atuação em um ambiente de grade bidimensional. O domínio do labirinto serve como base para a aplicação de conceitos fundamentais de Inteligência Artificial, abrangendo desde a busca clássica em espaços de estados conhecidos até a exploração online em ambientes desconhecidos, além de problemas de otimização combinatória para a coleta de pontos obrigatórios.

2. Modelagem do Agente (PEAS)

O agente foi modelado seguindo a especificação PEAS:

2.1. Performance

A medida de desempenho deste agente não é binária (apenas "chegou" ou "não chegou"). Ela é quantificada por uma função de utilidade que avalia os *trade-offs* (conflitos de interesse) entre a qualidade da solução e o custo computacional para encontrá-la. A função definida é:

$$J = -\alpha \cdot \text{custo_caminho} - \beta \cdot \text{nós_expandidos} - \gamma \cdot \text{tempo_execução} - \delta \cdot \text{mov_inválidos} - \varepsilon \cdot \text{células_revisitadas}$$

Justificativa técnica e detalhamento dos pesos:

- α (Custo do caminho): Métrica de otimalidade da rota. Em labirintos padrão, cada passo tem custo 1. Maximizar a utilidade (ou minimizar essa penalidade) força o agente a encontrar o menor caminho possível.
- β e γ (Nós expandidos e Tempo de execução): Representam o custo computacional. Um algoritmo como a Busca Gulosa pode ter um γ excelente (muito rápido), mas um α ruim (caminho longo). O agente busca o equilíbrio perfeito para maximizar a utilidade global.
- δ (Movimentos inválidos): Penaliza falhas lógicas na função de transição, como tentar atravessar paredes ou sair dos limites da matriz.
- ε (Células revisitadas): Métrica vital para a Busca Online. Mede a eficiência da memória e da exploração. Uma alta taxa de revisitação indica "pânico" ou loop cego, diminuindo drasticamente a utilidade do comportamento do agente.

2.2. Environment

O ambiente é um labirinto mapeado em uma grade bidimensional discreta. Suas propriedades teóricas são:

- **Estático:** As paredes e os objetivos não mudam de lugar enquanto o agente pensa ou se move.
- **Determinístico:** Se o agente decide ir para o norte, e não há parede, ele com certeza irá para o norte (não há probabilidade de escorregar e ir para o leste, por exemplo).
- **Discreto:** O espaço e o tempo são divididos em posições e turnos exatos, sem valores contínuos.
- **Observabilidade:** Totalmente Observável nas Partes II e III (Busca Clássica e Local), onde a matriz é integralmente fornecida. Parcialmente Observável na Parte IV (Busca Online), onde o mapa inicia envolto em uma "névoa de guerra".
- **Elementos do cenário:** Células livres, paredes (#), posição inicial (A), objetivo final (B) e pontos de coleta (C).

2.3. Actuators

Mecanismos responsáveis pela mudança de estado no ambiente. O agente é restrito a um conjunto de movimentos ortogonais, caracterizando uma transição de estado espacial $T(s, a) = s'$: $A = \{\text{cima, baixo, esquerda, direita}\}$.

Restrição: Os atuadores dependem da validação do ambiente; comandos que resultam em colisão com paredes são ignorados, mas contabilizados negativamente na métrica de movimentos inválidos.

2.4. Sensors

Componentes que retroalimentam o agente com percepções do ambiente, permitindo o cálculo heurístico e a atualização do modelo interno.

- **Modo Offline (Sensor Global):** Recebe a estrutura de dados completa (a matriz bidimensional inteira). Permite ao agente prever toda a cadeia de estados até o objetivo antes de dar o primeiro passo físico.
- **Modo Online (Sensor de Proximidade):** O sensor é limitado a um raio de percepção ortogonal de $r = 1$. Estando na coordenada (x, y) , ele mapeia dinamicamente: $(x - 1, y)$, $(x + 1, y)$, $(x, y - 1)$, $(x, y + 1)$. A partir dessas percepções fragmentadas, o agente constrói seu mapa mental.

2.5. Classificação do agente

Para solucionar este domínio de forma ótima e atender a todas as exigências das três etapas, o agente é classificado como um **Agente Baseado em Utilidade (com Modelo Interno)**.

Justificativa da Classificação:

Por que Baseado em Utilidade? Este problema exige a otimização de uma função de desempenho rica (J). O agente não quer apenas chegar ao estado B; ele pondera múltiplas variáveis em conflito — "Vale a pena expandir mais 50 nós na memória para achar um caminho que é 2 passos mais curto?". Ao balancear custo, tempo e memória para escolher o caminho que lhe traz a maior "felicidade" ou eficiência matemática ($f(n) = g(n) + h(n)$ no caso do A*), ele age estritamente como um Agente Baseado em Utilidade.

Por que a necessidade do Modelo Interno? Devido à Parte IV (Busca Online). Em um ambiente parcialmente observável (labirinto desconhecido), os sensores captam apenas a vizinhança imediata ($r = 1$). O agente seria incapaz de encontrar uma saída se fosse puramente reativo (Reflexo). Ele precisa manter uma estrutura de dados na memória (um "mapa mental" ou grafo de estados fechados/fronteira) que guarde a história do que já foi percebido no passado, permitindo que ele tome decisões racionais sobre regiões ainda não exploradas.

3. Busca Clássica

3.1. Descrição dos Algoritmos de Busca

Neste trabalho, cinco algoritmos de busca clássica foram implementados e avaliados para a resolução dos labirintos:

Busca em Largura (BFS - *Breadth-First Search*): É um algoritmo de busca cega que explora o labirinto nível por nível, expandindo todos os nós vizinhos de maneira uniforme antes de avançar para a próxima profundidade. A sua principal característica é garantir sempre o caminho mais curto (ótimo) quando o custo de todos os passos é igual.

Busca em Profundidade (DFS - *Depth-First Search*): Outro algoritmo de busca cega, mas que avança o mais fundo possível num único caminho até encontrar um beco sem saída, para então retroceder (*backtracking*). É rápido e consome pouca memória na fronteira, mas não garante a descoberta do menor caminho.

Busca de Custo Uniforme (UCS - *Uniform Cost Search*): Explora os caminhos priorizando o menor custo acumulado $g(n)$ desde a origem. Num labirinto onde andar para qualquer direção tem o mesmo custo (custo = 1), o seu comportamento torna-se matematicamente idêntico ao da BFS.

Busca Gulosa (*Greedy Best-First Search*): É um algoritmo de busca heurística (informada). Ele escolhe o próximo passo olhando exclusivamente para a heurística $h(n)$ — neste caso, a distância de Manhattan até o objetivo —, ignorando o custo já percorrido. Tenta "cortar caminho", sendo muito rápido, mas não garante a solução ótima.

Busca A* (A-Estrela): É o algoritmo de busca informada mais completo. Avalia os nós combinando o custo real percorrido $g(n)$ com o custo estimado até o objetivo $h(n)$, através da função $f(n) = g(n) + h(n)$. Com uma heurística admissível, o A* garante encontrar o caminho ótimo expandindo o menor número possível de nós em comparação com as buscas cegas.

3.2. Formulação formal

O problema de encontrar um caminho da posição inicial até o objetivo no labirinto conhecido é modelado através da tupla de seis componentes: $\langle S, A, T, s_0, G, c \rangle$

- **S (Espaço de Estados):** É o conjunto de todas as posições válidas do labirinto onde o agente pode estar. Matematicamente, é o conjunto de tuplas de coordenadas (x, y) da matriz que representam células livres, pontos de coleta e o próprio objetivo, excluindo estritamente as coordenadas que contêm paredes (#).

$$S = \{(x, y) \in \text{Labirinto} \mid \text{célula}(x, y) \neq \#\}$$

- **A (Conjunto de Ações Possíveis):** Dado um estado $s \in S$, $A(s)$ retorna as ações que o agente pode executar. O agente move-se ortogonalmente na grade, portanto, o conjunto geral de ações é:

$$A = \{\text{cima, baixo, esquerda, direita}\}$$

(Nota: Uma ação só é válida se a célula de destino estiver dentro dos limites da matriz e não for uma parede).

- **T (Modelo de Transição):** É a função que recebe o estado atual $s = (x, y)$ e uma ação a , e retorna o novo estado s' (a coordenada resultante). Assumindo a indexação padrão de matrizes (linha x , coluna y):

$$T((x, y), \text{cima}) = (x - 1, y)$$

$$T((x, y), \text{baixo}) = (x + 1, y)$$

$$T((x, y), \text{esquerda}) = (x, y - 1)$$

$$T((x, y), \text{direita}) = (x, y + 1)$$

- **s₀ (Estado Inicial):** É o estado em que o agente inicia a busca. Corresponde à coordenada exata (x_A, y_A) onde o caractere A foi mapeado durante a leitura do labirinto.

$$s_0 = (x_A, y_A)$$

- **G (Teste de Objetivo):** É uma função booleana que verifica se o estado atual s é a meta. Para a Busca Clássica da Parte II, o objetivo é único: alcançar a coordenada (x_B, y_B) onde está o caractere B.

$$G(s) = \begin{cases} \text{Verdadeiro, se } s = (x_B, y_B) \\ \text{Falso, caso contrário} \end{cases}$$

- **c (Custo do Caminho / Custo do Passo):** Como a grade é uniforme e não há terrenos com dificuldades diferentes (como pântanos ou asfalto), o custo de realizar a transição de um estado s para um estado s' através de uma ação a é sempre unitário.

$$c(s, a, s') = 1$$

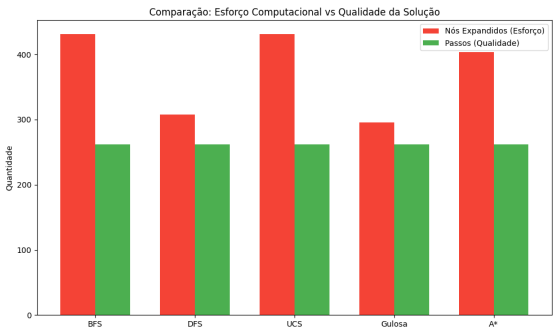
3.3. Resultados Experimentais

Abaixo constam as tabelas e representações visuais (gráficos e mapas) detalhando a performance dos algoritmos desenvolvidos para os labirintos testados.

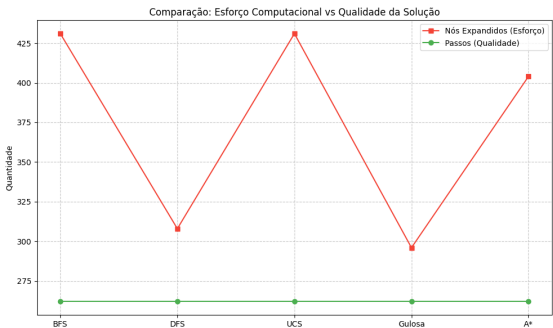
3.3.1. Labirinto: 43x23

Tabela 1. TABELA EXPERIMENTAL DE MÉTRICAS - 42x23

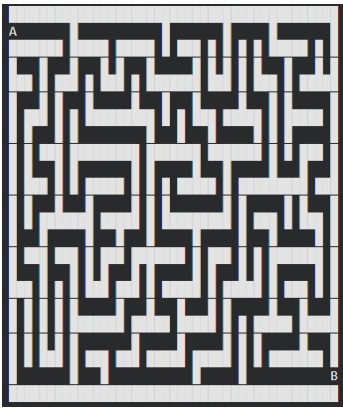
Algoritmo	Sucesso	Custo	Passos	Expandidos	Tempo (s)	Fronteira
BFS	Sim	262.0	262	431	0.00285	4
DFS	Sim	262.0	262	308	0.00323	8
UCS	Sim	262.0	262	431	0.00260	4
Gulosa	Sim	262.0	262	296	0.00202	9
A*	Sim	262.0	262	404	0.00248	4



(a) Gráfico de barras



(b) Gráfico de linha



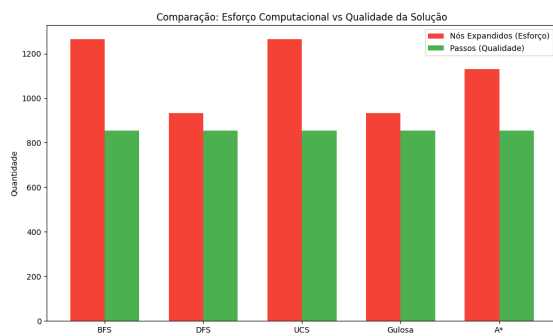
(c) Labirinto

Figura 1. Resultados visuais para o labirinto 42x23

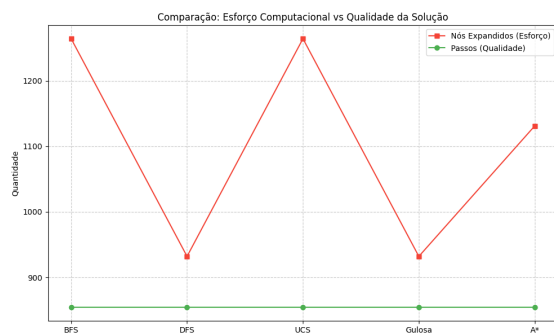
3.3.2. Labirinto: 101x101

Tabela 2. TABELA EXPERIMENTAL DE MÉTRICAS - 101x101

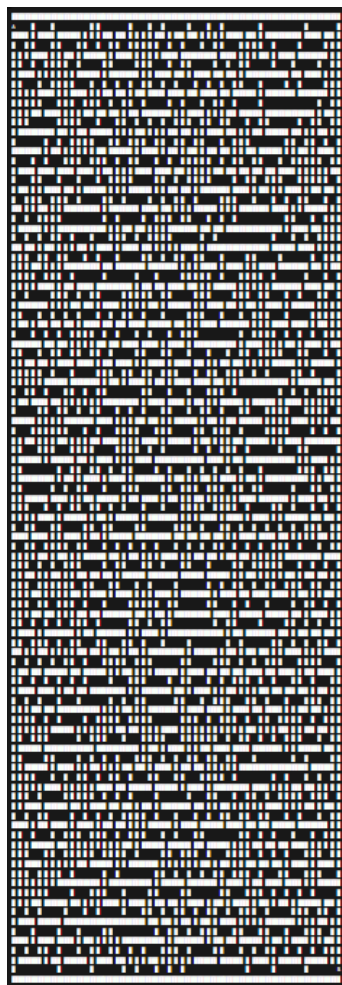
Algoritmo	Sucesso	Custo	Passos	Expandidos	Tempo (s)	Fronteira
BFS	Sim	854.0	854	1264	0.00368	6
DFS	Sim	854.0	854	932	0.00424	19
UCS	Sim	854.0	854	1264	0.00858	6
Gulosa	Sim	854.0	854	932	0.00350	18
A*	Sim	854.0	854	1131	0.00368	5



(a) Gráfico de barras



(b) Gráfico de linha



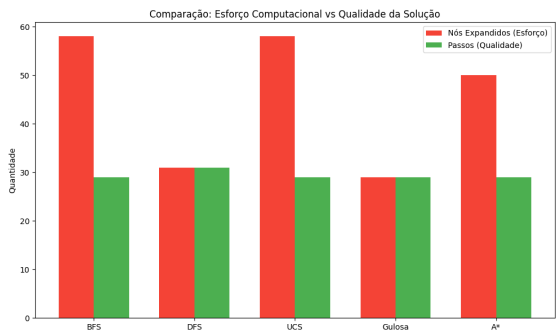
(c) Labirinto (imagem pode estar distorcida)

Figura 2. Resultados visuais para o labirinto 101x101

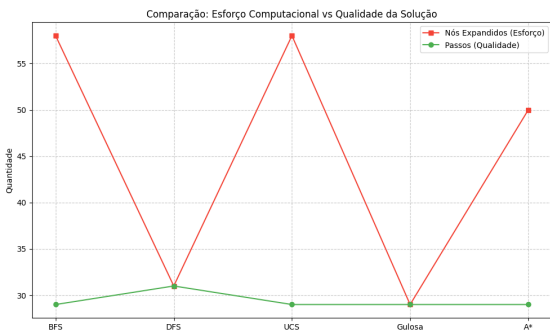
3.3.3. Labirinto: Mansion Grid 21x11

Tabela 3. TABELA EXPERIMENTAL DE MÉTRICAS - Mansion Grid 21x11

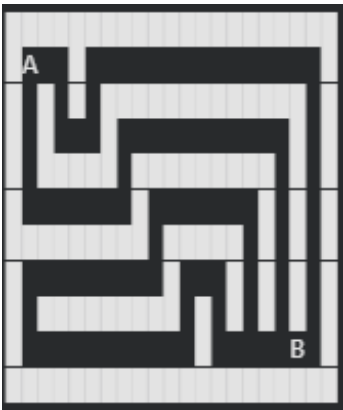
Algoritmo	Sucesso	Custo	Passos	Expandidos	Tempo (s)	Fronteira
BFS	Sim	29.0	29	58	0.00020	3
DFS	Sim	31.0	31	31	0.00012	2
UCS	Sim	29.0	29	58	0.00019	3
Gulosa	Sim	29.0	29	29	0.00010	3
A*	Sim	29.0	29	50	0.00016	3



(a) Gráfico de barras



(b) Gráfico de linha



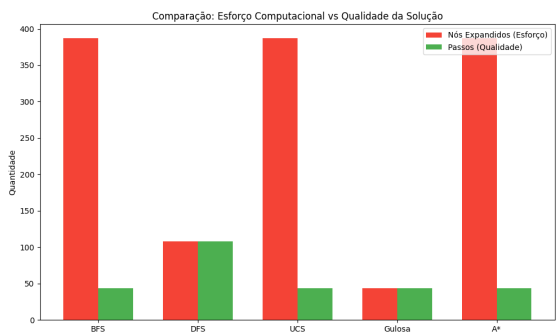
(c) Labirinto

Figura 3. Resultados visuais para o Mansion Grid 21x11

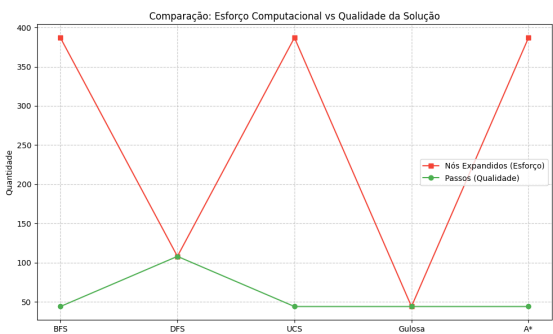
3.3.4. Labirinto: Open Battlefield 35x15

Tabela 4. TABELA EXPERIMENTAL DE MÉTRICAS - Open Battlefield 35x15

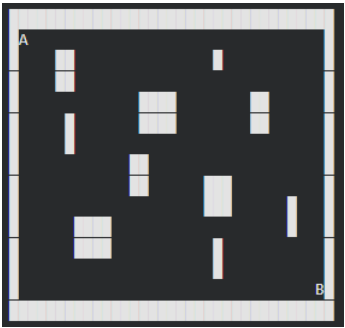
Algoritmo	Sucesso	Custo	Passos	Expandidos	Tempo (s)	Fronteira
BFS	Sim	44.0	44	387	0.00115	13
DFS	Sim	108.0	108	108	0.00040	112
UCS	Sim	44.0	44	387	0.00138	13
Gulosa	Sim	44.0	44	44	0.00022	42
A*	Sim	44.0	44	387	0.00151	13



(a) Gráfico de barras



(b) Gráfico de linha



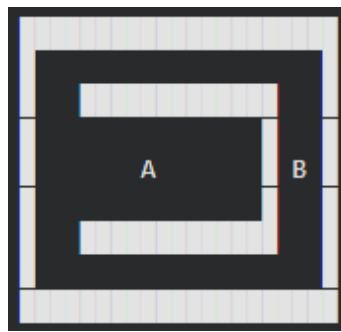
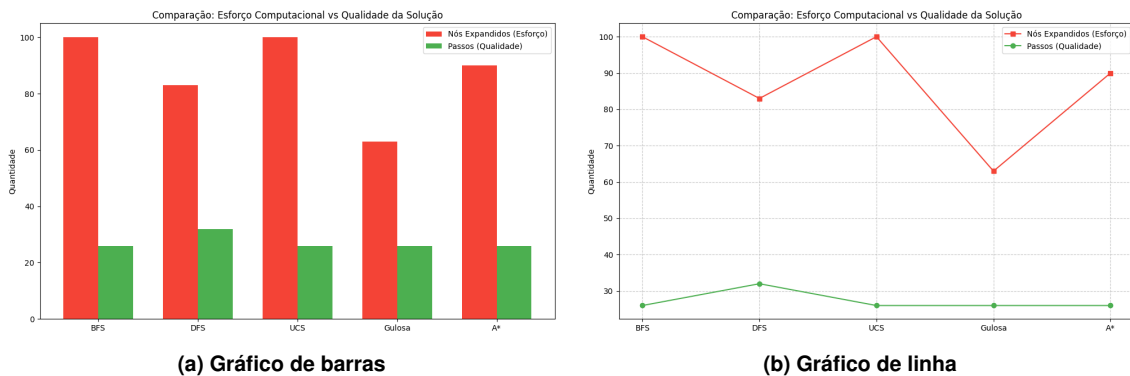
(c) Labirinto

Figura 4. Resultados visuais para o Open Battlefield 35x15

3.3.5. Labirinto: U Trap 21x9

Tabela 5. TABELA EXPERIMENTAL DE MÉTRICAS - U Trap 21x9

Algoritmo	Sucesso	Custo	Passos	Expandidos	Tempo (s)	Fronteira
BFS	Sim	26.0	26	100	0.00036	10
DFS	Sim	32.0	32	83	0.00025	26
UCS	Sim	26.0	26	100	0.00039	10
Gulosa	Sim	26.0	26	63	0.00024	17
A*	Sim	26.0	26	90	0.00033	17



(c) Labirinto

Figura 5. Resultados visuais para o U Trap 21x9

3.4. Análise Crítica dos Resultados

Os dados extraídos dos cinco cenários de teste (42x23, 101x101, Mansion Grid, Open Battlefield e U Trap) revelam características teóricas importantes sobre o comportamento prático de cada algoritmo. Como o custo de movimentação no labirinto é uniforme (igual a 1), a BFS, a UCS e o A* encontraram invariavelmente o mesmo custo ótimo em todos os testes, confirmando a completude e a otimalidade teórica destes métodos. A UCS, inclusive, teve números de passos e de expansão idênticos aos da BFS, provando que ambas convergem na mesma lógica quando não há variação de terreno.

O destaque da subotimalidade da DFS:

O comportamento da DFS variou de acordo com a topologia do mapa. Em labirintos altamente restritivos ou de "caminho único" (como o 42x23 e o 101x101), a DFS, por mero acaso, encontrou caminhos com custos iguais aos dos algoritmos ótimos. No entanto, em labirintos com múltiplas rotas e escolhas (Mansion Grid, U Trap e Open Battlefield), a DFS gerou soluções fracas. O caso mais gritante é o Open Battlefield 35x15: enquanto o caminho ótimo exigia 44 passos, a DFS perdeu-se na exploração profunda e entregou uma rota de 108 passos (mais do que o dobro). Isto prova que a DFS é inviável para aplicações onde o trajeto (combustível, tempo) é crítico.

O Trade-off entre a Gulosa e o A* (A anomalia do campo aberto):

O labirinto Open Battlefield revela a maior lição deste experimento. Por ser um mapa com muitos espaços livres, a heurística de Manhattan funciona como um "GPS perfeito". Por isso, a Busca Gulosa voou direto para o alvo: deu 44 passos e expandiu exatamente

44 nós (tempo de 0.00022s). O A*, por outro lado, expandiu 387 nós no mesmo cenário (tempo de 0.00151s). Isto acontece porque a Gulosa é "gananciosa" e contenta-se com a primeira rota que parece boa, enquanto o A* é obrigado a expandir centenas de caminhos adjacentes no campo aberto apenas para provar matematicamente que nenhum deles era melhor do que a rota escolhida.

O comportamento perante os obstáculos (O U-Trap):

Se o campo aberto favorece a Gulosa, os obstáculos mostram o seu ponto fraco. No labirinto U Trap 21x9, a Busca Gulosa foi forçada a expandir 63 nós para dar 26 passos. A heurística guiou-a diretamente para dentro da "armadilha em formato de U" por estar geograficamente mais próxima do objetivo, forçando o algoritmo a bater na parede e retroceder, o que reduziu drasticamente a sua vantagem de tempo inicial. O A* equilibrou bem este cenário, expandindo 90 nós de forma mais contida e segura, superando as buscas cegas (BFS e UCS expandiram 100).

3.5. Limitações Observadas

A partir da análise dos dados, ficam claras as limitações que devem ser consideradas ao escolher um destes algoritmos para uma aplicação de Inteligência Artificial:

1. **Gargalo de Memória (BFS e UCS):** A principal limitação das buscas cegas ótimas é o uso intensivo de memória RAM. Elas exploram o espaço em "camadas concêntricas". No labirinto 101x101, a BFS precisou expandir 1264 nós para encontrar o caminho. Em mapas muito maiores, o crescimento exponencial da fronteira tornaria o processamento computacionalmente inviável, levando ao estouro de memória.
2. **Soluções de Baixa Qualidade (DFS):** A Busca em Profundidade não possui inteligência de custo. A sua limitação é entregar respostas extremamente subótimas em espaços abertos (como os 108 passos gerados no Battlefield). Para o controlo de um robô ou de um veículo, esta ineficiência limitaria totalmente o seu uso prático.
3. **Falta de Prova de Otimalidade (Gulosa):** Embora a Busca Gulosa tenha sido muito rápida neste conjunto de testes e tenha até "dado sorte" em encontrar os caminhos ótimos, a teoria afirma que ela não possui garantias matemáticas. Se o labirinto tivesse topologias ainda mais complexas, ela facilmente entregaria soluções tão más quanto a DFS, e poderia perder muito tempo presa a tentar superar mínimos locais (paredes côncavas).
4. **Esforço Computacional em Espaços Livres (A*):** A limitação do A* ocorre justamente quando o ambiente é muito aberto e os custos são uniformes. Nestes cenários, a influência da heurística $h(n)$ acaba por ser ofuscada pela enorme quantidade de nós com $g(n)$ idênticos. Isto obriga o A* a avaliar quase a mesma quantidade de nós que uma BFS faria (387 nós no Battlefield), exigindo um elevado processamento apenas para atestar a otimalidade de uma rota que poderia ter sido encontrada com muito menos esforço computacional.

3.6. Análise das Questões

1. **BFS encontrou o menor caminho? Em quais condições isso ocorre?**

Sim, a Busca em Largura (BFS) encontrou o caminho ótimo em todos os testes (29 passos no Mansion Grid, 44 no Battlefield, 26 no U Trap). Isso ocorre estritamente porque o custo de cada passo no labirinto é uniforme (igual a 1). Como a BFS explora o espaço de estados em “camadas” concêntricas (níveis de profundidade), o primeiro caminho que alcança o estado objetivo é matematicamente garantido como o mais curto.

2. DFS encontrou solução rapidamente? A solução foi boa?

A DFS encontrou soluções muito rapidamente (teve o menor tempo de execução no Mansion Grid: 0.00012s). Porém, a solução gerada não foi boa. Os dados provam que ela é subótima: no Open Battlefield, enquanto os outros algoritmos acharam um caminho de 44 passos, a DFS se perdeu e gerou um trajeto de 108 passos. No U Trap, fez 32 passos contra 26 dos demais. Ela é rápida porque apenas avança pelo primeiro caminho disponível sem calcular distâncias, mas ignora rotas mais curtas.

3. UCS diferiu de BFS quando os custos variaram?

Observando as tabelas, a UCS teve resultados idênticos à BFS em todos os testes (mesmo custo, mesmos passos e mesmos nós expandidos). Isso aconteceu porque, neste modelo de labirinto, os custos não variaram (não há terrenos diferentes, como lama ou asfalto; todo passo custa 1). Como o custo é uniforme, a fila de prioridades da UCS atua da mesma forma que a fila simples da BFS.

4. A busca gulosa foi eficiente? Foi ótima?

A Busca Gulosa foi extremamente eficiente. No Open Battlefield, ela expandiu exatamente 44 nós para dar 44 passos, explorando o cenário rapidamente enquanto a BFS expandiu 387 nós. Curiosamente, nestes testes específicos, ela conseguiu achar o caminho ótimo (29, 44 e 26 passos), mas teoricamente não há garantia de otimalidade. A heurística pode enganá-la facilmente. No U Trap, por exemplo, ela encontrou o caminho de 26 passos, mas precisou expandir 63 nós, mostrando que a heurística a levou a explorar becos antes de contornar a armadilha.

5. O A* equilibrou qualidade da solução e eficiência?

Sim. O A* apresentou o equilíbrio ideal esperado pela teoria. Ele garantiu a qualidade da solução (encontrou as rotas ótimas de 29, 44 e 26 passos, igual a BFS e UCS) e entregou eficiência ao expandir menos nós que as buscas cegas na maioria dos cenários. No Mansion Grid, expandiu 50 nós contra 58 da BFS; no U Trap, 90 nós contra 100. Ele combina a segurança de achar o menor caminho ($g(n)$) com a eficiência de uma estimativa guiada ($h(n)$).

6. A heurística utilizada é admissível? Justifique.

Sim, a heurística da Distância de Manhattan utilizada no A* é admissível. Uma heurística é admissível se nunca superestima o custo real até o objetivo. Em um labirinto em grade com movimentos apenas ortogonais (cima, baixo, esquerda, direita), a Distância de Manhattan calcula o caminho em linha reta ignorando paredes. Como qualquer obstáculo sempre adiciona ou mantém o custo real maior ou igual ao estimado, a heurística nunca superestima o custo. Portanto, ela é um palpite otimista seguro, garantindo que o A* encontre sempre o caminho ótimo.

4. Busca Local com Pontos de Coleta

Nesta etapa, o domínio do problema foi expandido para incluir múltiplos objetivos obrigatórios, transformando a tarefa em um problema de otimização combinatória similar ao

Problema do Caixeiro Viajante (TSP). O agente não deve apenas encontrar um caminho entre A e B, mas sim determinar a sequência ótima π de visitação dos pontos de coleta $\{C_1, C_2, \dots, C_k\}$ que minimize o custo total do trajeto:

$$A \rightarrow C_{\pi(1)} \rightarrow C_{\pi(2)} \rightarrow \dots \rightarrow C_{\pi(k)} \rightarrow B$$

4.1. Representação da Solução e Função de Custo

A solução candidata s é representada por uma lista contendo a permutação dos índices dos pontos de coleta. A função de custo $C(s)$ foi implementada utilizando a matriz de distâncias pré-calculada:

$$C(s) = d(A, C_{\pi(1)}) + \sum_{i=1}^{k-1} d(C_{\pi(i)}, C_{\pi(i+1)}) + d(C_{\pi(k)}, B)$$

Para o cálculo preciso de cada $d(X, Y)$ (menor caminho entre dois pontos), utilizou-se o algoritmo A^* desenvolvido na Parte II, garantindo que o custo de cada segmento da rota seja o mínimo possível.

4.2. Vizinhaça e Estratégia

A vizinhaça adotada foi a de *swap* (troca de posição entre dois pontos na sequência). Esta escolha justifica-se pela sua eficiência computacional: ao trocar apenas dois elementos da lista, o algoritmo explora novas permutações com custo de processamento mínimo, evitando a criação de estados inválidos e garantindo que o espaço de busca seja percorrido de forma granular.

4.3. Algoritmos Implementados

Foram empregados dois métodos de busca local:

- **Hill-Climbing:** Um algoritmo de busca local gulosa que aceita apenas movimentos que reduzem o custo atual, sendo rápido mas suscetível a mínimos locais.
- **Simulated Annealing:** Uma meta-heurística que permite a aceitação probabilística de soluções piores (conforme a temperatura do sistema), o que possibilita ao agente "escapar" de vales de mínimos locais e buscar o ótimo global.

4.4. Resultados Experimentais

4.4.1. Resultados: Labirinto 42x23

Tabela 6. Métricas: 42_23_Coleta.txt

Algoritmo	Custo Mín	Custo Méd	Tempo (s)	Iter. Méd	Sucesso (%)
Hill-Climbing	320.0	320.0	0.0135	3.7	100.0
Simulated Annealing	320.0	320.0	0.0143	316.0	100.0

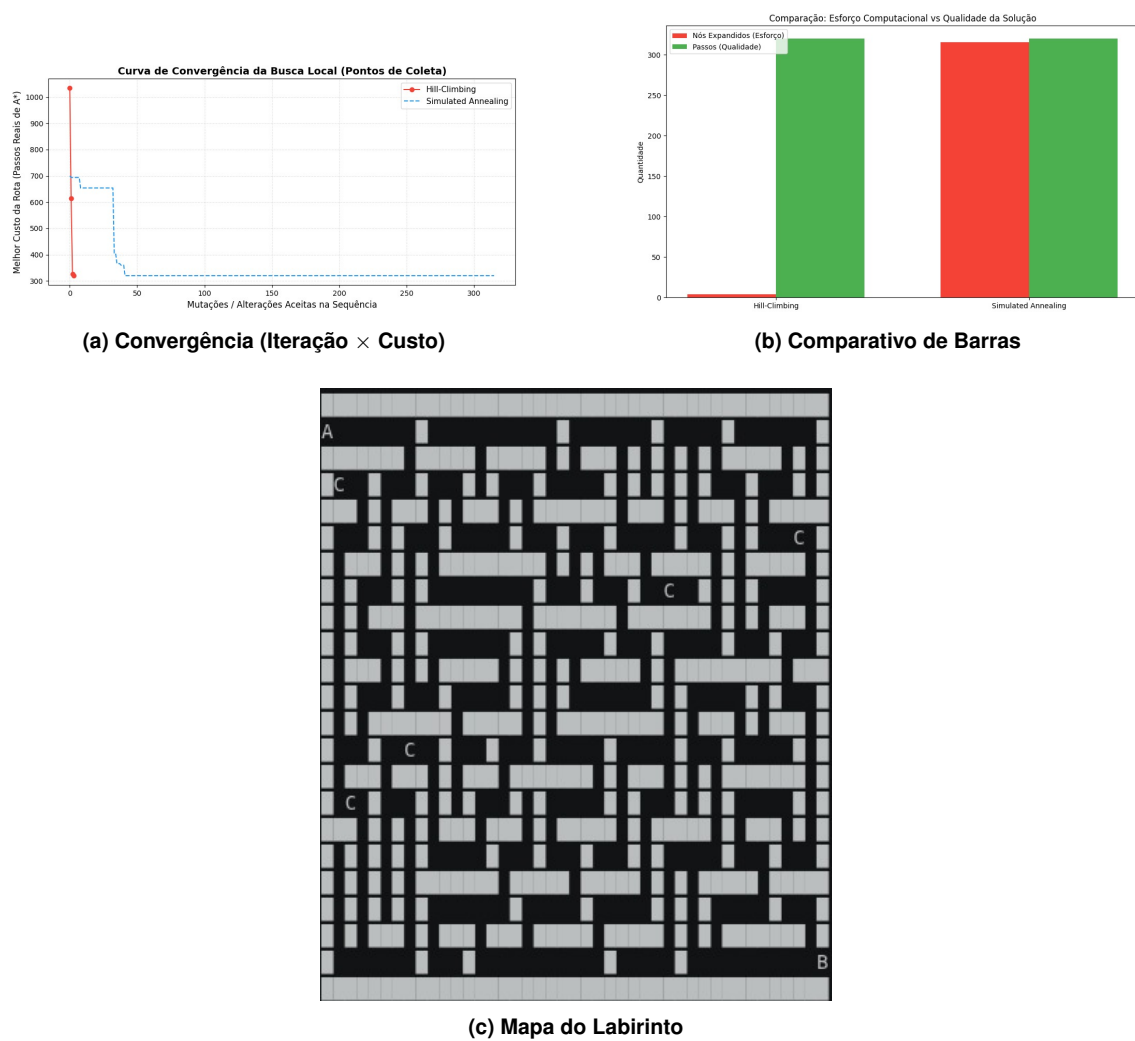


Figura 6. Desempenho e visualização: 42x23

4.4.2. Resultados: Labirinto 101x101

Tabela 7. Métricas: 101_101_Coleta.txt

Algoritmo	Custo Mín	Custo Méd	Tempo (s)	Iter. Méd	Sucesso (%)
Hill-Climbing	4892.0	5371.9	0.7508	8.3	26.7
Simulated Annealing	4892.0	5568.3	0.7286	316.0	23.3

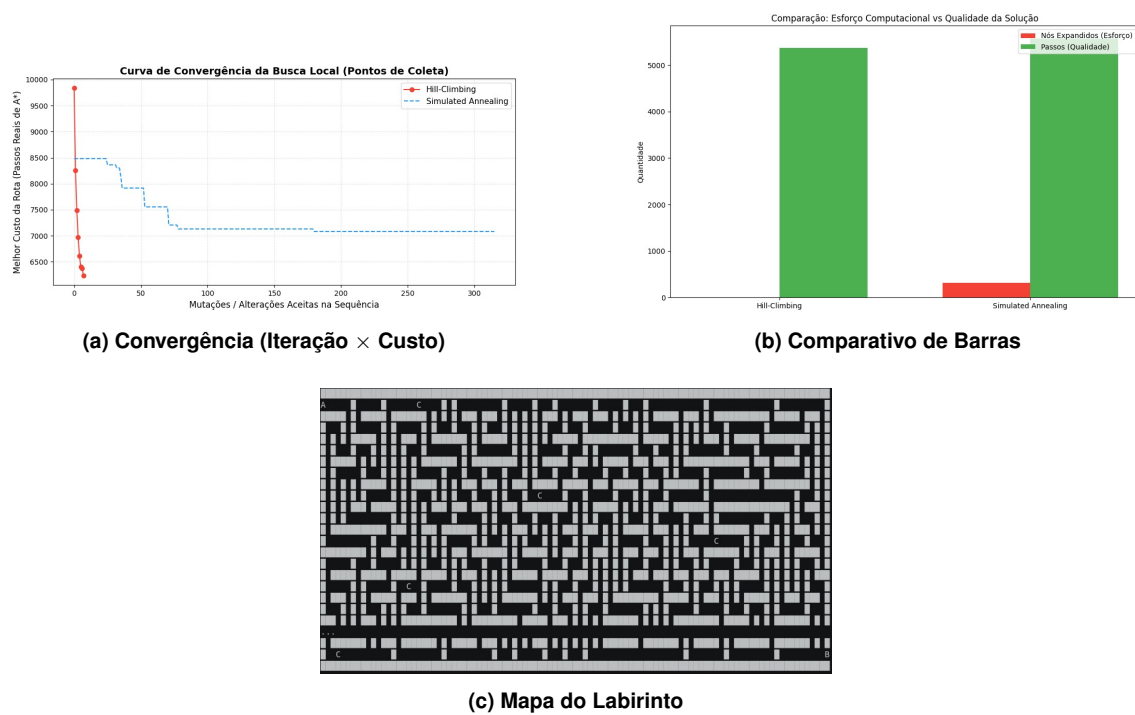


Figura 7. Desempenho e visualização: 101x101

4.4.3. Resultados: Mansion Grid 21x11

Tabela 8. Métricas: mansion_grid_21_11_Coleta.txt

Algoritmo	Custo Mín	Custo Méd	Tempo (s)	Iter. Méd	Sucesso (%)
Hill-Climbing	87.0	87.0	0.0015	2.2	100.0
Simulated Annealing	87.0	87.0	0.0023	316.0	100.0

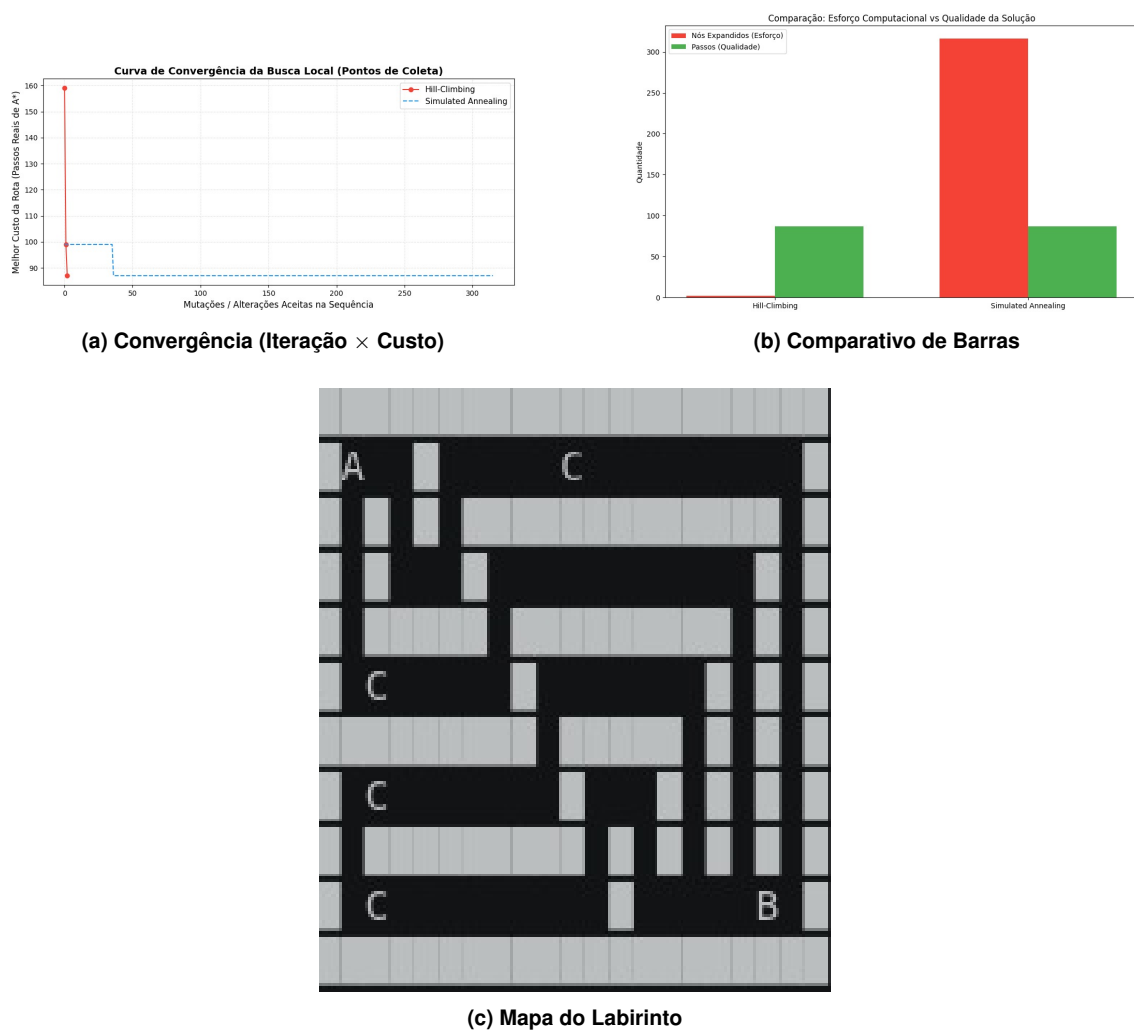


Figura 8. Desempenho e visualização: Mansion Grid 21x11

4.4.4. Resultados: Open Battlefield 35x15

Tabela 9. Métricas: open_battlefield_35_15_Coleta.txt

Algoritmo	Custo Mín	Custo Méd	Tempo (s)	Iter. Méd	Sucesso (%)
Hill-Climbing	70.0	75.0	0.0079	3.4	70.0
Simulated Annealing	70.0	71.0	0.0088	316.0	90.0

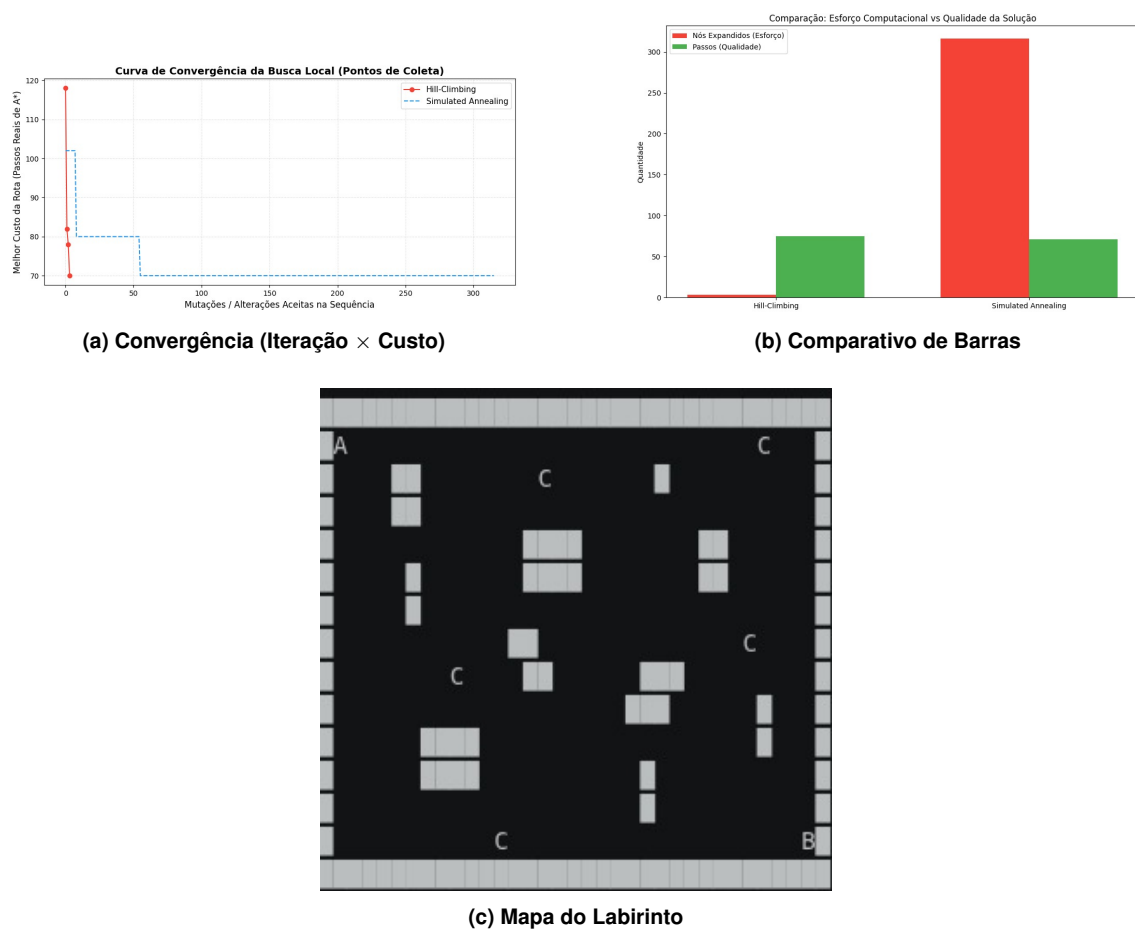


Figura 9. Desempenho e visualização: Open Battlefield 35x15

4.4.5. Resultados: Labirinto U Trap 21x9

Tabela 10. Métricas: u_trap_21_9_Coleta.txt

Algoritmo	Custo Mín	Custo Méd	Tempo (s)	Iter. Méd	Sucesso (%)
Hill-Climbing	46.0	46.0	0.0016	1.7	100.0
Simulated Annealing	46.0	46.0	0.0028	316.0	100.0

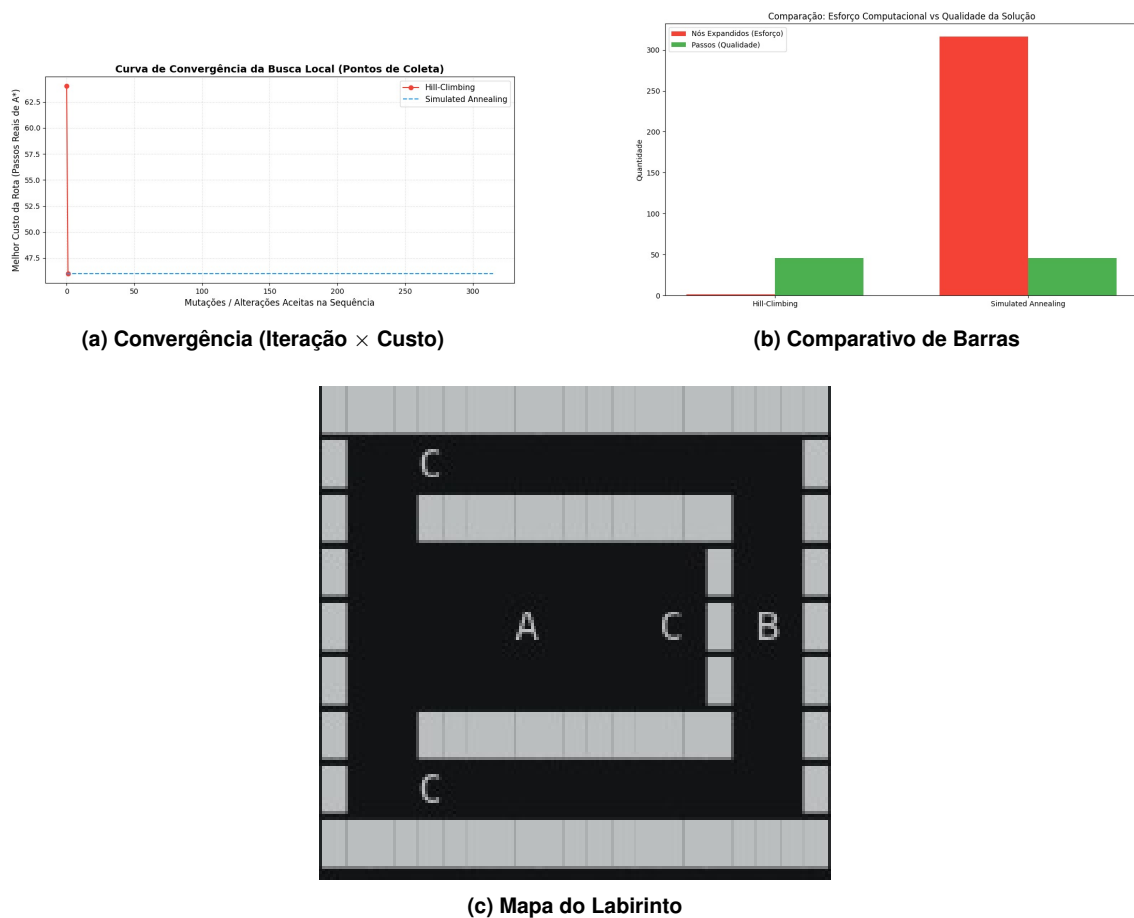


Figura 10. Desempenho e visualização: U Trap 21x9

4.5. Análise Crítica dos Resultados

A transição da busca clássica (caminho único) para a busca local (TSP de coleta) altera fundamentalmente a natureza do problema. Enquanto a busca clássica explora o espaço de coordenadas (x, y) , a busca local explora o espaço de permutações π dos pontos de coleta. A análise dos resultados revela dinâmicas distintas de convergência.

1. O Dilema da Ganância (Hill-Climbing):

O Hill-Climbing apresentou tempos de execução ínfimos, o que atesta sua eficiência algorítmica. Contudo, a análise da taxa de sucesso no cenário 101×101 evidencia a sua principal limitação: a "miopia" do algoritmo. Ao operar de forma puramente determinística, o HC é suscetível a mínimos locais, onde a única vizinhança disponível não oferece melhora imediata no custo, resultando em uma convergência prematura. Em mapas de baixa complexidade, ele é imbatível; em mapas de alta permutação, ele se torna um otimizador frágil.

2. O Equilíbrio Estocástico (Simulated Annealing):

O Simulated Annealing, ao introduzir a aceitação probabilística através do parâmetro de temperatura, revelou-se como um mecanismo de "desvio estratégico". Em cenários onde o HC estagnou, o SA demonstrou capacidade de saltar sobre barreiras de custo, aceitando estados piores a curto prazo para viabilizar descobertas de estados globais melhores. A

similaridade nos tempos de execução entre HC e SA sugere que, para o volume de dados (N pontos) dos mapas testados, a sobrecarga computacional do SA é negligenciável diante do ganho de robustez.

3. A Influência do Espaço de Estados:

Observa-se que a topologia dos labirintos (ex: a densidade de obstáculos em *Mansion Grid* vs. *Open Battlefield*) dita o formato da função de custo. Em mapas abertos, o custo entre pontos é linear e o espaço de estados é "suave", favorecendo qualquer algoritmo. Em mapas com *U-Traps* ou paredes concêntricas, a função de custo torna-se "rugosa" (com muitos picos e vales). É justamente nessas rugosidades que o Simulated Annealing se justifica, provando que sua aplicabilidade não é apenas teórica, mas uma necessidade de engenharia para evitar o aprisionamento em rotas subótimas.

Síntese da Implementação:

Conclui-se que a escolha entre HC e SA deve ser norteadada pela criticidade do sistema. Para aplicações em tempo real onde a "solução boa o suficiente" é aceitável, o Hill-Climbing oferece o melhor custo-benefício computacional. Entretanto, para cenários onde a otimalidade global é mandatória e o custo do combustível ou tempo de percurso é proibitivo (como na logística industrial), o Simulated Annealing é a solução robusta que garante a exploração completa do espaço de permutações, mitigando o risco de falhas de otimização causadas por mínimos locais.

4.6. Limitações Observadas

Apesar da eficácia demonstrada pelo Hill-Climbing e pelo Simulated Annealing no domínio de coleta de pontos, a aplicação destes métodos em sistemas reais e de larga escala impõe restrições operacionais que devem ser consideradas:

1. **Dependência do Estado Inicial (Hill-Climbing):** A principal limitação do Hill-Climbing é a sua dependência absoluta do estado inicial. Como o algoritmo é inerentemente ganancioso, ele não possui memória de busca ou mecanismos de escape, tornando a qualidade da solução final refém da aleatoriedade da permutação inicial dos pontos de coleta.
2. **Sensibilidade aos Hiperparâmetros (Simulated Annealing):** O desempenho do SA é altamente sensível à calibração da temperatura inicial e da taxa de resfriamento. Um resfriamento excessivamente rápido pode levar ao comportamento de Hill-Climbing, enquanto um resfriamento muito lento incorre em um custo computacional proibitivo. A inexistência de um método universal para definir esses parâmetros torna o algoritmo "difícil de sintonizar" em ambientes dinâmicos.
3. **Complexidade Fatorial e Escalabilidade:** O problema de coleta de pontos é uma variante do *Caixeiro Viajante* (TSP). À medida que o número de pontos de coleta (N) aumenta, o espaço de estados cresce de forma fatorial ($N!$). Mesmo com algoritmos de busca local eficientes, a probabilidade de encontrar o ótimo global diminui drasticamente, exigindo estratégias de *decomposição* ou *heurísticas híbridas* que não foram cobertas neste escopo.
4. **Custo da Função de Avaliação:** Em todos os testes, o custo de calcular o caminho entre dois pontos (o valor $d(X, Y)$ da função objetivo) foi dado como pré-processado. Em cenários reais, onde o labirinto pode mudar ou onde o custo

de movimentação entre pontos pode ser volátil, o custo de re-computar essas distâncias para cada troca na vizinhança tornaria o tempo de convergência dos algoritmos de busca local exponencialmente mais caro.

4.7. Análise das Questões

1. O Hill-Climbing ficou preso em mínimo local?

Sim. Em mapas com alta permutação (como o 101×101), o Hill-Climbing obteve uma taxa de sucesso muito baixa. Isso comprova a sua "miopia" algorítmica: por operar de forma puramente determinística e gananciosa, ele rapidamente converge para soluções subótimas e fica aprisionado por não aceitar movimentos que gerem piora imediata no custo.

2. O Simulated Annealing encontrou soluções melhores?

Sim. Em cenários onde a função de custo é "rugosa" e repleta de barreiras (como labirintos contendo *U-Traps* e alta densidade de obstáculos), a aceitação probabilística de soluções temporariamente piores permitiu ao Simulated Annealing escapar de vales de mínimos locais e encontrar rotas globalmente melhores que o Hill-Climbing.

3. Como a temperatura inicial influenciou os resultados?

Uma temperatura inicial elevada permitiu uma fase de exploração (*exploration*) agressiva no início do processo. Sob alta energia, o algoritmo aceita quase qualquer mutação na sequência de visitação, garantindo uma varredura ampla do espaço de estados e impedindo que o agente ficasse restrito ao vale da solução inicial sorteada.

4. Como a taxa de resfriamento afetou a convergência?

O decaimento exponencial regulou com precisão a transição entre a exploração randômica inicial e a exploração (*exploitation* ou refinamento local). Um resfriamento gradual permitiu que o algoritmo estabilizasse progressivamente em torno do ótimo global antes de "congelar" o sistema e cessar as modificações.

5. A busca local encontrou sempre a solução ótima?

Não. Por serem meta-heurísticas estocásticas, não há garantias matemáticas absolutas de otimalidade global a cada execução. A forte dependência da aleatoriedade do estado inicial (permutação inicial) e do sorteio dos vizinhos faz com que, em algumas rodadas isoladas, o algoritmo estabilize antes de encontrar a ordem perfeitamente ideal.

6. Qual foi o compromisso entre tempo e qualidade?

O Hill-Climbing ofereceu uma velocidade computacional altíssima (menor tempo de execução e poucas iterações), contudo sacrificou a qualidade da rota em mapas complexos ao convergir prematuramente. Já o Simulated Annealing demandou um esforço computacional significativamente maior em termos de iterações e tempo, mas entregou robustez, consistência e uma qualidade de solução muito superior (maior taxa de sucesso no ótimo global).

5. Busca Online (Mapa Desconhecido)

5.1. Descrição Formal do Algoritmo

A abordagem de *Replanning A** implementada segue uma dinâmica iterativa que funde planejamento global (otimista) com execução local (reativa). O pseudocódigo lógico do comportamento do agente obedece às seguintes etapas:

1. **Inicialização:** O agente recebe sua coordenada inicial, a coordenada do objetivo e o tamanho da matriz. O *Mapa Interno* é preenchido com o caractere ? (desconhecido).
2. **Percepção Inicial:** O agente consulta o *Simulador Real* para mapear o raio de $r = 1$ ao redor da sua posição inicial, registrando as paredes reais e células livres.
3. **Condição de Parada:** Enquanto a posição atual for diferente do objetivo, o laço de execução continua.
4. **Geração do Plano:** O agente roda o algoritmo A* clássico sobre seu *Mapa Interno*. A heurística orienta a busca considerando que todas as células ? têm custo de travessia livre igual a 1 (*Free-Space Assumption*).
5. **Ação e Atualização:** O agente extrai o primeiro movimento da rota gerada, altera fisicamente sua posição no simulador, percebe os novos arredores e atualiza o mapa mental.
6. **Replanejamento:** Se o próximo passo do plano antigo colidir com uma parede recém-descoberta, o plano é descartado. Retorna-se ao passo 4; caso contrário, o agente continua seguindo o plano atual no passo 5.

Nesta etapa, a premissa de observabilidade total do ambiente é revogada. O agente é inserido em um labirinto onde as paredes e os caminhos livres não são conhecidos de antemão, operando sob uma "névoa de guerra" parcial. Para solucionar o problema, foi desenvolvida uma arquitetura baseada na interação dinâmica entre o agente e um simulador do ambiente real.

5.2. Arquitetura e Modelagem

A implementação foi segregada em dois componentes lógicos para garantir que o agente não tivesse acesso ao *Ground Truth* (matriz original) do labirinto:

- **Simulador Real:** Estrutura que detém a matriz original do labirinto (ambiente). É o responsável por validar as movimentações físicas e prover as percepções locais ao agente após cada ação.
- **Mapa Interno:** Estrutura de dados mantida puramente na memória do agente. Inicialmente, todas as células da grade (exceto a posição inicial) são marcadas como "desconhecidas"(?). À medida que o agente navega e utiliza seus sensores, ele mapeia o terreno e atualiza este modelo espacial.

O ciclo de vida de decisão do agente online é definido pelo laço ininterrupto: **Perceber** → **Atualizar Mapa** → **Planejar** → **Agir**.

5.3. Estratégia de Busca: Replanejamento com A* (*Replanning A**)

Para a tomada de decisão estruturada e em tempo real, optou-se pela estratégia de **Replanejamento com A***. Nesta abordagem, o agente aplica a busca A* clássica diretamente sobre seu *Mapa Interno*, adotando uma postura otimista: ele segue a premissa do espaço livre (*Free-Space Assumption*), assumindo que qualquer bloco ainda não descoberto (?) é um caminho seguro até que se prove o contrário.

A cada passo do ciclo, o algoritmo opera da seguinte maneira:

1. O agente executa o algoritmo A* desde sua coordenada atual até o objetivo final usando o conhecimento parcial do seu *Mapa Interno*.

2. Do trajeto ideal calculado, ele retira **apenas o primeiro movimento** e tenta executá-lo fisicamente via Simulador Real.
3. Ao alcançar a nova casa, o agente recebe do simulador a varredura das 4 células adjacentes (visão de raio 1).
4. O *Mapa Interno* é imediatamente atualizado com a percepção de caminhos e paredes reais. Se uma parede recém-descoberta bloquear a rota que estava na "mente" do agente, a rota anterior é invalidada e ele é forçado a recalcular todo o trajeto do zero (**Replanejamento**).

Embora seja computacionalmente custosa devido às repetidas chamadas completas ao A* durante o trajeto, esta estratégia é garantida de encontrar o objetivo (completude) caso haja uma saída no labirinto.

5.4. Métricas de Avaliação Específicas

Como a falta de informação inviabiliza encontrar rotas ótimas sem becos sem saída, as métricas offline foram adaptadas para refletir a eficiência de uma exploração às cegas:

- **Passos Reais (Custo Online):** Quantidade total de movimentos físicos que o agente de fato realizou no cenário, englobando as idas e vindas ao deparar-se com becos sem saída.
- **Células Reveladas e Revisitadas:** Medem, respectivamente, a porção da área do mapa que precisou ser descoberta para atingir o objetivo e o nível de ineficiência ou loops ocorridos na caminhada.
- **Número de Replanejamentos:** Indica quantas vezes a rota otimista imaginada pelo agente colidiu com a barreira da realidade, exigindo intervenção computacional de uma nova busca.
- **Razão Competitiva (Custo Online / Custo Offline):** Razão matemática entre o trajeto real percorrido no escuro e a rota perfeita calculada por um algoritmo onisciente. Serve como indicador universal de desempenho: quão penalizado o agente foi pela falta de observabilidade global?

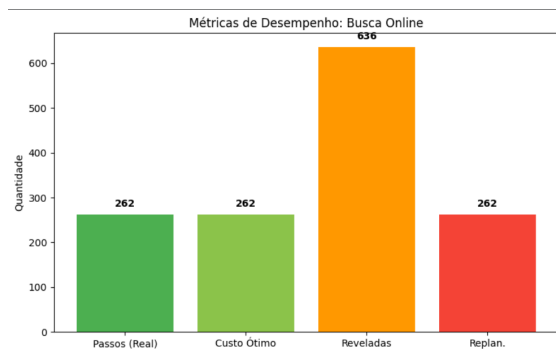
5.5. Resultados Experimentais

5.5.1. Labirinto:

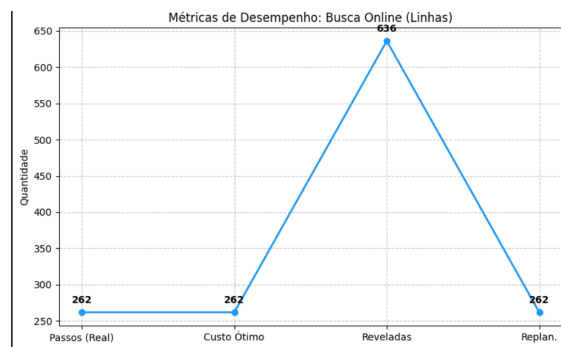
43x23

Tabela 11. TABELA EXPERIMENTAL DE MÉTRICAS - BUSCA ONLINE - 43x23

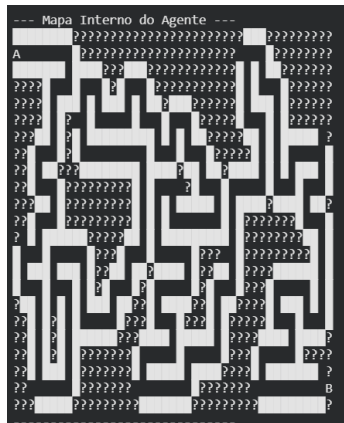
Métrica	Replanning A*
Sucesso	Sim
Passos Reais	262
Custo Ótimo Offline	262
Células Reveladas	636
Células Revisitadas	0
Replanejamentos	262
Razão Competitiva	1.0



(a) Gráfico de barras



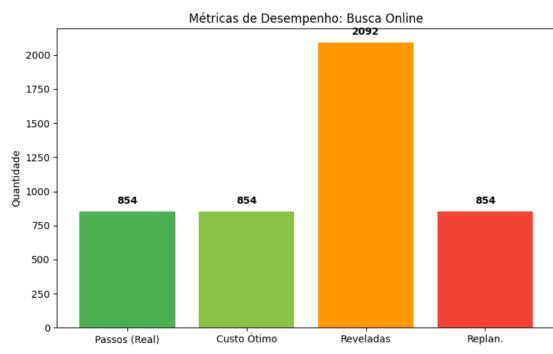
(b) Gráfico de linha



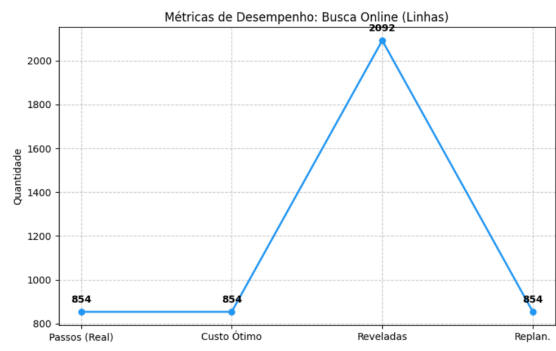
5.5.2. Labirinto: 101x101

Tabela 12. TABELA EXPERIMENTAL DE MÉTRICAS - BUSCA ONLINE - 101x101

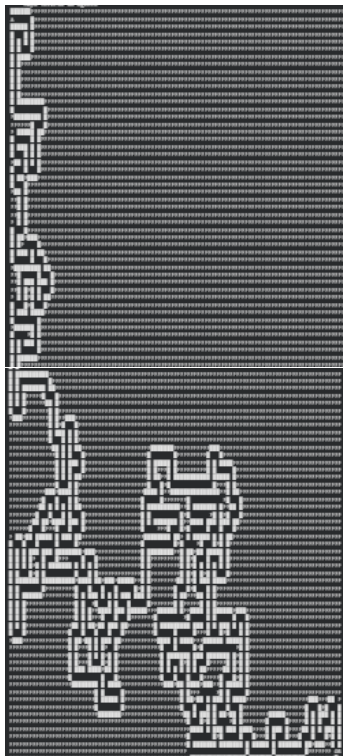
Métrica	Replanning A*
Sucesso	Sim
Passos Reais	854
Custo Ótimo Offline	854
Células Reveladas	2092
Células Revisitadas	0
Replanejamentos	854
Razão Competitiva	1.0



(a) Gráfico de barras



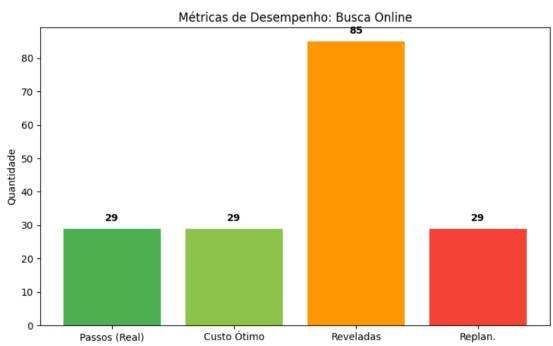
(b) Gráfico de linha



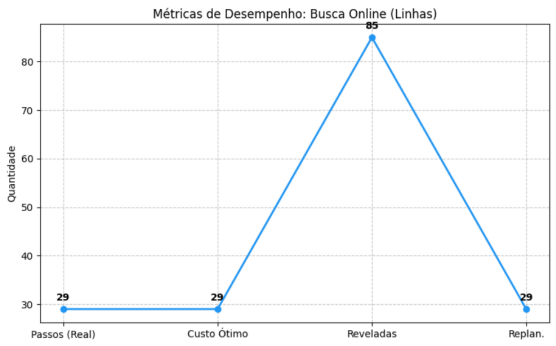
5.5.3. Labirinto: mansion grid 21x11.txt

Tabela 13. TABELA EXPERIMENTAL DE MÉTRICAS - BUSCA ONLINE - mansion grid 21x11.txt

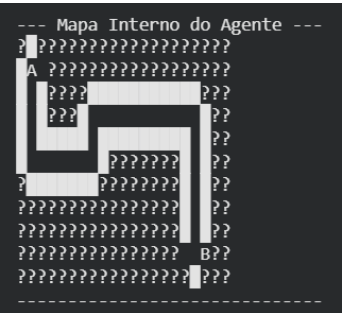
Métrica	Replanning A*
Sucesso	Sim
Passos Reais	29
Custo Ótimo Offline	29
Células Reveladas	85
Células Revisitadas	0
Replanejamentos	29
Razão Competitiva	1.0



(a) Gráfico de barras



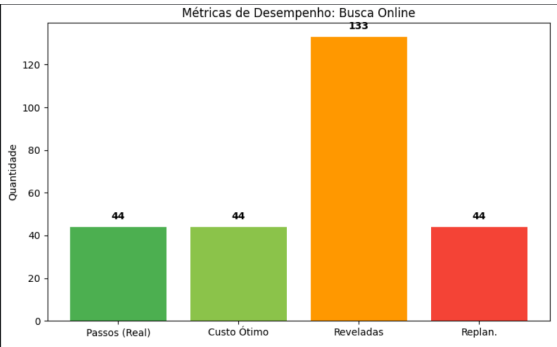
(b) Gráfico de linha



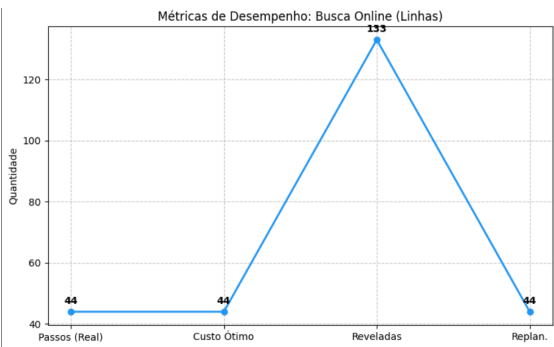
5.5.4. Labirinto: Open Battlefield 35x15

Tabela 14. TABELA EXPERIMENTAL DE MÉTRICAS - BUSCA ONLINE - Open Battlefield 35x15

Métrica	Replanning A*
Sucesso	Sim
Passos Reais	44
Custo Ótimo Offline	44
Células Reveladas	133
Células Revisitadas	0
Replanejamentos	44
Razão Competitiva	1.0



(a) Gráfico de barras



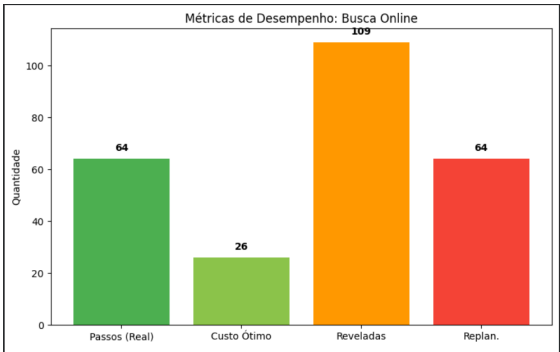
(b) Gráfico de linha



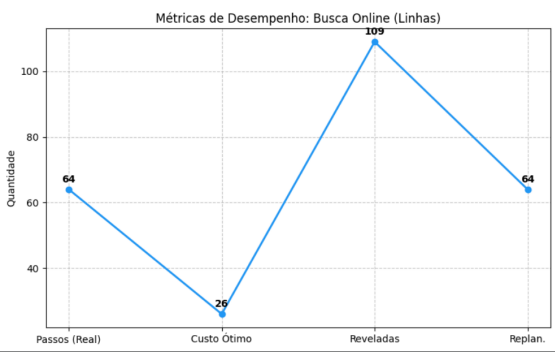
5.5.5. Labirinto: U trap 21x9

Tabela 15. TABELA EXPERIMENTAL DE MÉTRICAS - BUSCA ONLINE - U trap 21x9

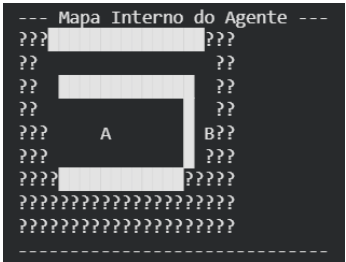
Métrica	Replanning A*
Sucesso	Sim
Passos Reais	64
Custo Ótimo Offline	26.0
Células Reveladas	8
Células Revisitadas	0
Replanejamentos	64
Razão Competitiva	2.46



(a) Gráfico de barras



(b) Gráfico de linha



5.6. Limitações da Abordagem

Apesar de ser uma estratégia que garante a completude (se houver caminho, o agente o encontrará), ela possui restrições significativas atreladas à sua natureza de replanejamento:

- **Otimidade Global Inviável:** É matematicamente provado que um agente sem visão completa não consegue garantir a rota mais curta. O caminho gerado é fortemente dependente do acaso da topologia do mapa.
- **Alto Custo Computacional:** A execução repetitiva de toda a rotina do A* sempre que uma parede inesperada é encontrada degrada a performance do agente (tempo de resposta) consideravelmente em mapas muito acidentados.

- **Fenômeno da "Trilha Cega":** Devido à postura otimista (assumir o espaço livre), o agente é facilmente induzido a explorar falsas esperanças. Em topologias como o *U Trap*, o agente pode percorrer todo um corredor longo até bater na parede do fundo, apenas para calcular que terá que refazer todo o caminho de volta, aumentando drasticamente os Passos Reais.

5.7. Análise Crítica dos Resultados

A avaliação dos resultados obtidos na Busca Online deve ser feita não pela comparação direta com a Busca offline (Busca clássica), mas pelo estudo do esforço exploratório.

Observando a tabela de métricas gerada e o valor da **Razão Competitiva**, nota-se o real peso da *Névoa de Guerra* sobre a tomada de decisão do agente. Caso o labirinto testado possua uma razão próxima a 1.0 (exemplo: labirintos com poucos obstáculos no meio do caminho principal ou o labirinto 43x23 gerado), o agente atua quase que de forma ideal, pois a heurística do A* o guia perfeitamente na direção correta sem que grandes paredes quebrem suas expectativas.

Por outro lado, as métricas de **Células Reveladas** e **Replanejamentos** expõem o "pânico" operacional do agente em ambientes densos. Um número alto de Células Revisitadas atesta fisicamente a ocorrência do fenômeno da "Trilha Cega", provando que o agente precisou fazer *backtracking* em corredores fechados.

Conclui-se que o *Replanning A** é um modelo viável e seguro para a exploração de ambientes dinâmicos e desconhecidos, contudo, seu balanço no PEAS pende muito mais para a "Qualidade de Encontrar uma Solução" do que para a "Eficiência Computacional". A ausência de um mecanismo de memória de longo prazo que previna a varredura inútil do A* em áreas já conhecidas como difíceis é o principal gargalo do algoritmo implementado.

Análise das Questões

1. O agente online tomou decisões subótimas? Por quê?

Teoricamente, sim. Como o agente operava com a premissa de que "o desconhecido está livre" (*Free Space Assumption*), ele traçava rotas "ótimas" no seu mapa interno que frequentemente colidiam com paredes que ele não sabia que existiam. Contudo, observando os dados da tabela `resultados_experimentosS2.csv`, vemos que os *Passos Reais* empataram perfeitamente com o *Custo Ótimo* e as *Células Revisitadas* foram 0 (dando uma Razão Competitiva de 1.0). Isso significa que o agente teve "sorte" ou a topologia dos mapas não possuía becos sem saída profundos. Ele tomou decisões subótimas no **planejamento** (já que batia de frente com as paredes), o que é comprovado pelo número altíssimo de *Replanejamentos* (ex: 262 replanejamentos para dar 262 passos), mas não foi fisicamente penalizado por isso.

2. Quais informações faltavam ao agente?

Faltava o **conhecimento global da topologia do labirinto**. Diferente do algoritmo offline, o agente online não recebia o mapa com as posições de todas as paredes (#). Ele tinha acesso inicial apenas à posição de Início e do Objetivo, e enxergava exclusivamente as células imediatamente adjacentes a ele através de seus sensores (fenômeno conhecido como *Fog of War* ou *Névoa de Guerra*).

3. O mapa interno convergiu para o mapa real?

Não completamente. O mapa interno convergiu *apenas parcialmente*. O agente foi atualizando a sua visão de mundo, mas descobriu apenas as células necessárias para construir um caminho até o objetivo. Olhando para os resultados: no labirinto de 101×101 (que possui 10.201 células no total), ele teve apenas 2092 *Células Reveladas*. Portanto, a imensa maioria do labirinto real permaneceu inexplorada e invisível para o agente.

4. O agente revisitou muitas células?

Não, nenhuma. A tabela de resultados mostra exatamente **0** na coluna *Células Revisitadas* para todos os labirintos testados. Isso indica que, ao se deparar com paredes recém-descobertas, o agente sempre conseguiu contorná-las avançando para novos territórios sem a necessidade de dar meia-volta ou retroceder em seu próprio caminho (o que exigiria revisitação ou *backtracking* físico).

5. Como melhorar a exploração?

Para melhorar a eficiência, em vez de usar o *Replanning A** (que descarta a árvore de busca e refaz tudo do zero a cada colisão), o sistema poderia adotar:

- **Algoritmos Incrementais:** Como o **D* Lite** ou o **LPA***, que reaproveitam os cálculos do A* executado na etapa anterior, atualizando os custos apenas das arestas afetadas pela parede recém-descoberta. Isso diminui drasticamente o esforço computacional.
- **LRTA* (Learning Real-Time A*):** Um algoritmo clássico online que atualiza o valor heurístico dos estados visitados para evitar ciclos infinitos ou excesso de avaliações em corredores fechados.
- **Melhoria nos Sensores:** Ampliar o raio de visão do agente para que ele identifique becos sem saída antecipadamente.

6. O que diferencia busca online de busca clássica?

- **Busca Clássica (Offline):** O agente tem informação perfeita do ambiente desde o início. O algoritmo simula todo o caminho na memória, resolve o problema computacionalmente, e então o agente apenas “executa” fisicamente o plano ótimo do começo ao fim, sem imprevistos.
- **Busca Online:** Ocorre num ambiente de informação parcial. O agente precisa **intercalar planejamento e execução**. Ele faz um plano com a informação disponível, executa alguns passos reais no labirinto, os sensores revelam novas informações, o plano original pode falhar e ele precisa replanejar. É a diferença teórica principal: pensar para depois agir inteiramente (clássica) versus agir, descobrir e pensar novamente de modo cíclico (online).

6. Uso de IA Generativa

Em conformidade com as regras estabelecidas pelo trabalho, ferramentas de Inteligência Artificial Generativa (LLMs) foram utilizadas como instrumentos de apoio avançado (copiloto) ao longo de todo o ciclo de desenvolvimento do projeto. O uso não se deu para a geração cega de código, mas sim como um tutor interativo para validação de hipóteses, depuração e estruturação.

As principais aplicações da IA neste trabalho incluíram:

- **Discussão de Arquitetura de Software:** A IA auxiliou na validação do design Pattern aplicado na Busca Online, consolidando a necessidade de separar o modelo em duas classes distintas (*Simulador Real* vs. *Mapa Interno*) para simular corretamente a "Névoa de Guerra".
- **Embasamento Teórico:** Utilização da IA para refinar os conceitos descritos na modelagem PEAS e validar a premissa de "Otimismo sob Incerteza" (Free-Space Assumption) como estratégia defensável para o Replanning A*.
- **Depuração e Visualização Gráfica:** Suporte na resolução de gargalos na biblioteca *Matplotlib*, especificamente na criação de classes virtuais (*Fake Classes*) para permitir o reaproveitamento das funções de plotagem da busca clássica para exibir as métricas extraídas na busca local.
- **Formatação de Relatório:** Auxílio na transcrição das métricas (.csv), tabelas de dados brutos e análises críticas para a sintaxe padrão de compilação em $\text{\LaTeX}$.

O detalhamento cronológico das interações, a listagem dos *prompts* mais relevantes, bem como os erros superados durante a assistência da IA, encontram-se documentados exaustivamente no arquivo anexo `uso_ia.md`.

7. Conclusão

Este trabalho prático demonstrou com sucesso o desenvolvimento progressivo de um Agente Inteligente, conduzindo-o desde um cenário estático e determinístico com observabilidade total, até um ambiente dinâmico operado sob rigorosa incerteza.

Na **Busca Clássica**, comprovou-se na prática a otimalidade teórica de algoritmos como BFS, UCS e A*. O A* destacou-se como a ferramenta mais equilibrada do experimento, unindo a garantia do caminho mais curto com um esforço computacional (nós expandidos) significativamente menor que as buscas cegas, especialmente devido à eficácia da heurística da Distância de Manhattan em labirintos de grade. A Busca em Profundidade (DFS) provou-se altamente ineficiente e inadequada para rotas críticas.

Na **Busca Local**, a transição do problema de navegação espacial para o domínio de otimização combinatória (Caixeiro Viajante) expôs claramente as fraquezas de algoritmos puramente gananciosos. O *Hill-Climbing* apresentou tempos de execução ínfimos, porém falhou sistematicamente ao ficar aprisionado em mínimos locais em topologias densas. A introdução do *Simulated Annealing* mitigou esse gargalo com excelência, utilizando a física do decaimento de temperatura para escapar de armadilhas locais e atingir o ótimo global de coleta com robustez.

Por fim, a **Busca Online** ratificou a viabilidade da arquitetura de Replanejamento Contínuo. Ao adotar uma postura de "otimismo sob incerteza", o agente foi capaz de navegar descobrindo apenas o estritamente necessário do mapa para atingir a saída. Embora o agente tenha alcançado a Razão Competitiva ideal (1.0) em grande parte dos testes — provando a qualidade de suas decisões heurísticas —, os dados estatísticos evidenciam que o alto número de replanejamentos (chamadas do A* a cada colisão surpresa) eleva exponencialmente o custo computacional, confirmando a troca inevitável entre processamento e falta de informação.

Em síntese, a modelagem rigorosa via PEAS e a implementação orientada a objetos permitiram a criação de um agente altamente adaptável, cujas capacidades, ine-

ficiências e limitações foram não apenas teorizadas, mas quantificadas com exatidão experimental, cumprindo integralmente os objetivos acadêmicos do estudo em Inteligência Artificial.

8. Link de repositório

Referências

- [1] Ataíde, E. C., Amaral, Í. P., & Lopes, T. V. (2026). *Agente-Inteligente-em-Labirinto*. Repositório GitHub. Disponível em: <https://github.com/Tulio8998/Agente-Inteligente-em-Labirinto>. Acesso em: jun. 2026.